FACT0RN
Integer Factorization as Proof of Work

Escanor Liones

May 27, 2022

Abstract

Blockchain technology was introduced in 2009 by Satoshi Nakamoto with Bitcoin. The security of this technology is determined by the amount of work done to validate transactions in the system. Any would be attacker would have to redo this work as well as all subsequent work done in the system to modify any given transaction, making it ever more computationally expensive to edit the system ledger. Satoshi introduced the mechanism, known as Proof of Work (PoW), by which this work is done and verified. New mechanisms have been developed since, including: Proof of Stake (Algorand), Proof of History(Solana), Proof of Storage(Siacoin), among several others. I would like to introduce a new blockchain with PoW securing transactions, just like bitcoin, but replacing the work component by factoring integers instead of hashing. In addition, one more new idea is introduced to the blockchain space: a deadpool for the proof of work. The FACT0RN Blockchain's development has been funded in its entirety by Coinbase through their 2021 Crypto Community Fund program. [10]

Keywords. FACTORN, factorn, Blockchain, PoW, Factoring, Coinbase, Coinbase Grant, Crypto Community Fund, Deadpool.

1 Introduction

The sheer amount of planetary resources dedicated to mining in the form of hashing SHA2 incentivized by the financial aspects of blockchain technology has raised concerning questions among members of every community that understands the astronomical numbers associated to the resource usage devoted to this one endeavor. An analysis done in late 2021 by the New York Times on the electricity usage of the Bitcoin network indicated that the lowest electricity consumption estimate was on par with the total consumption of Washington State for a year — and more than 7 times as much as Google's global operations [8].

These concerns would be alleviated if mining, currently in the form of hashing, advanced other areas of humanity in the process of "securing the ledger".

I claim that such a thing is possible. In particular, that by replacing hashing with a problem that is the subject of active research today we can advance knowledge areas that would ultimately benefit the world beyond just the ledger. Which means that while securing the ledger energy and resources would simultaneously be spent on this research area.

The core work of bitcoin, performing SHA2, is not an active area of research outside the cryptocurrency mining communities, whereas breaking integers apart into their prime factors is. Let us incentivize computational problems that have been open and the subject of research for millennia, centuries, or at worst decades in the hope that the financial aspect associated with blockchain technology will be able to fund advancement in these fields.

The resources allocated to mining SHA2 have accelerated advancements of knowledge in two areas: hardware design in the form of ASICs and Data Center architecture [1]. While these two areas are important I think we can do vastly better; that is, by designing a block chain whose PoW would require, not just better hardware but better mathematical theory, and in so doing, advancing knowledge in mathematics, cryptography and security in a way we can all benefit.

2 Integer Factorization

One of the oldest problems in mathematics we learned about in elementary school. Among other things, we learned how to factor integers into their prime factors. It has remained an open problem whether factoring can be done efficiently or not; in modern formal terms we ask whether factoring is in P or not, where P is the class of computational problems that can be solved in polynomial time. Only in the past 400 years, since Pierre de Fermat and Leonhard Euler, have significant advancements been made in factoring algorithms, but the question remains open.

By way of analogy, factoring numbers in elementary school is difficult, if you are in elementary school. The first few days of factoring two digit numbers are easy for those who learned their times table, but even for them factoring three digit numbers is difficult. The more digits there are the more difficult it is, and so most school systems stop at three digit numbers. However, once we get to high school and learn algebra we can use brand new tricks to factor three digit numbers with ease without having to have the time tables memorized for three digit numbers. If we go on to college and take a number theory course, then a whole suite of computationally cheap tricks become available to us and we can factor 3 digit numbers doing less work still.

The factoring algorithms in existence today are the same in this regard. There are several of them. Each one is king for a certain number of digits and then another algorithm becomes more efficient for the next range, until we get to the open waters and the best algorithm we have for classical non-quantum computers can be used. This algorithm, known as Number Field Sieve (NFS) [5], still has sub-exponential complexity in time and hence is not in P as far as we know. But, it is unknown if there exists an algorithm better than NFS whose computational complexity is polynomial in time using traditional non-quantum computers.

It turns out that in general factoring numbers with hundreds of digits is really difficult. So difficult in fact that the security of banks, Fortune 500 companies, online payments, and governments depend on it. The RSA cryptographic system [9] is based on how difficult it is to factor integers into their prime factors and how easy it is to check if such prime factors have been found; hard to find, easy to check. RSA is widely used today.

There are several communities that factor as a hobby. Concretely at least 14 such projects can be named, see reference [11] for a link that lists all these projects. Several of these projects are looking for primes of a special form (e.g Mersenne primes, brilliant numbers, etc), others are looking for sequences of a special form like Aliquot sequences. The Cunningham Project looks to factor integers of a particular form and has existed since 1925; yes, since 1925. The mersenneforum is one place where several of these different projects gather and discuss improved factoring methods.

These communities are important because they can help the blockchain be more secure by factoring and getting rewarded, but also because they can submit numbers to be factored using their rewards, giving a monetary incentive for folks who would traditionally not care about factoring to factor. This environment is likely conducive to animate investment in mathematical research given the incentive structure of the blockchain. It is a symbiotic relationship between the cryptographic community, the mathematics community, the blockchain community and the finance community.

The RSA factoring challenge[4] was announced on March 18, 1991 and ended in 2007 by RSA Laboratories. It awarded the cash dollar bounties on numbers factored quarterly. The largest cash prize awarded before the challenge ended was $\$20,000$ dollars to Jens Franke for factoring RSA-640 on November 2, 2005. That is to say, the FACT0RN blockchain would be, in essence, the 21-st century version of the RSA Challenge.

There will be a Tip Jar wallet whose funds will be used for the following purposes:

1. Fund the technical development of the FACT0RN blockchain.
2. Fund mathematical research.
3. Fund development of related projects: e.g. factoring software and factoring hardware.

Tip Jar Address: fact1qln3xayyyuhwww5jl6c4xtf8e3aqzym6wl45rrv

Symbol	Significance
$\lvert \cdot \rvert_2$	Number of binary digits
$\lvert\lvert \cdot \rvert\rvert_2$	$\lvert \cdot \rvert_2$ applied twice
$\lvert\lvert$	Concatenation
$\tilde{n}$	Search range from random seed
$\lceil \cdot \rceil$	Round up to nearest integer
$\lfloor \cdot \rfloor_t$	Round down to nearest multiple of t
$(T)_r$	Merkle Tree root of T
$\lvert \cdot \rvert$	Magnitude of a number, or cardinality of a set
p_1, p_2	Prime numbers such that $p_1 < p_2$

Figure 1: Symbols used in the next section.

3 Blockchain Overview

Covering every aspect of the blockchain in detail would make this whitepaper unbearably long. The content will be limited to guiding principles and some detail where needed for better understanding without diving into the weeds.

A blockchain based on factoring as PoW does not yet exist, it seems, because there does not exist a way to deterministically generate semiprimes with 'good' factors, i.e. of about the same number of digits, without first knowing the factors. Note that the PoW is not generating semiprimes, but rather factoring them. This subtle distinction is the key to make FACT0RN possible.

The way to address the issue is to make this problem part of the solution in a very particular way. Because it is hard to find such semiprimes, make them part of the PoW by way of factoring and allow miners to be rewarded for finding them. This is to say, allow miners to find strong semiprimes but in such a way integer factorization is the computationally cheapest way to determine if a number is a semiprime instead of allowing primes to be generated and then multiplied together. In fact, on average quite a few nonces and numbers will have to be factored before a strong semiprime is found by any one miner. [1]

Formally, the problem to be solved for the FACT0RN blockchain is the following:

$$\text{gHash(PrevHash, Merkle Tree Hash, nBits, version, time, nonce)} + wOffset = p_1 \cdot p_2$$

Figure 2: FACT0RN Equation.

where the following conditions hold:

1. $|\text{wOffset}| <= 16 \cdot nBits$
2. $|p_1 \cdot p_2|_2 = nBits$
3. $|p_1|_2 = |p_2|_2$
4. p_1, p_2 are primes under 50 rounds of Miller-Rabin and the Baillie–PSW Primality test.

A miner needs to find three parameters to submit a new valid block: a nonce, an offset and the smallest prime factor of a strong semiprime generated by the nonce and the offset; the smallest factor must be submitted or the block will be rejected. So, in essence for every block a miner has to find a nonce such that the nBit-bit number produced by gHash plus some offset within the constrain produces a strong semiprime.

Why must the smallest prime be submitted? The measure for work done for PoW uses an ECM based function to measure how much work was done in the factorization [2]. This function works based on the smallest factor found, hence the requirement to submit the smallest prime factor. This is the best proxy I was able to find to compute this quantity based on the smallest factor found even if the Number Field Sieve is used to factor.

[1] See Appendix A for a complete worked out example of mining one block.

Heuristically we can determine that we may demand the prime factors be both the same number of binary digits even if the resulting product has an odd number of binary digits. There is about a $60\% - 40\%$ split into even multiplicands and odd multiplicands, respectively, when the multipliers all have the same bitsize.

Given the interest there is in factoring in the cryptographic community as well as in over a dozen communities that factor as a hobby the FACT0RN blockchain is introducing a deadpool; any user can pay a fee in FACT0RN coins and submit a number to be factored to the deadpool. In addition, anyone else can add more coins to any number on the deadpool.

The reward function is exponential in shape but modest in coins rewarded. The exponential shape is required because the marginal cost of factoring a bigger number is also exponential; otherwise, the opportunity cost of factoring a smaller number will always be negative and it will be more profitable to find and factor smaller semiprimes.

While the shape is exponential the amount of coins rewarded must remain 'reasonably bounded'; otherwise, the inflation generated by that yearly influx of coins through the miners would adversely impact the coin's value.

Given the marginal cost of factoring a slightly bigger number, and the inflationary pressure of awarding brand new coins when factoring, there is a balancing act to be staged on the reward function only Cirque Du Soleil would have an easy time balancing. Nonetheless, a serious effort has been made to balance it all.

The design of this blockchain will be informed by four principles, in order of precedence: security, game theory, economics and heuristics. Which is to say, a heuristic parameter will be adjusted/changed/removed/replaced if there is a economics or game theoretical principle/theorem for it, and that will be adjusted/changed/removed/replaced if it negatively impacts security.

4 Technical Executive Summary (TES)

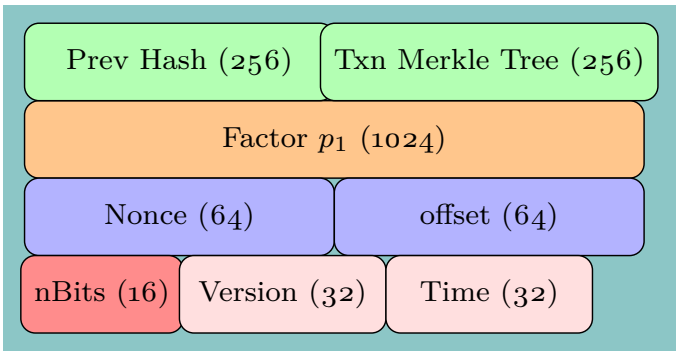

Figure 3: FACT0RN Block Header.

This block header is 1744-bits long, or 218 bytes; about 3.63 MB per year at target block time speed; p_1 and p_2 must be prime or the block is invalid, where gHash + offset $= p_1 \cdot p_2$. The number in parenthesis in Figure 3 is the bit length of each field.

Characteristic	Value
Symbol	FACT
Transaction Fee	10,000 Satoshis = 0.0001 FACT0RN Coins
Target Block Time	30 minutes
Block Size (soft limit)	20 MB
Block Size (hard limit)	80 MB
Block Reward Function	$R(N)$
Start Difficulty	$\lvert n\rvert_2 = \lvert p_1 \cdot p_2\rvert_2 = 230$
Hashing Block	SHA2
Other Hashing	SHA3 512, Scrypt, Blake2b and Whirlpool
Pseudo-random Source	gHash(Prev Hash | $(TXN)_r$) | nonce)
gHash	generates W
$\tilde{n}$	$16 \cdot \lvert W\rvert_2$
Feature 1	Factoring Deadpool

Figure 4: FACT0RN Blockchain characteristics.

The fundamental parameters of the blockchain are listed in Figure 4. Both gHash and $R(N)$ are addressed more in depth ahead.

$$R(N) = \begin{cases} \lfloor 11178600 \cdot 2^{\frac{\lvert p_1\rvert_2 + (\lvert p_1\rvert_2 \mod 2)}{32}} \rfloor_{1024} + 1023 & \text{if } \lvert p_1\rvert_2 = \lvert p_2\rvert_2 \\ 0 & \text{if } \lvert p_1\rvert_2 \neq \lvert p_2\rvert_2 \end{cases}$$

Figure 5: FACT0RN Blockchain Reward Function in Satoshis. N $= p_1 \cdot m$.

This function $R(N)$ is based on the big-O notation cost of the Number Field Sieve(NFS) to factor a number of bitsize $\lvert N\rvert_2$. It doubles the reward every time N adds 64 binary digits to its size. The constants associated with said difficulty, formally $L_n[\frac{1}{3}, \sqrt[3]{\frac{64}{9}}]$ [7], have been updated with two goals in mind: reduced floating point rounding errors by using powers of two and whole numbers, and dampening the exponential nature of the function with satoshi units instead of coin units. The rounding and adding 1023 is just a precaution against floating point arithmetic rounding errors. The modulus 2 addition pertains to the $40\% - 60\%$ split mentioned previously; it applies a slight premium on incentives due to scarce semiprimes when N is odd.

Figure 6 shows the expected size of the smallest prime for a strong semiprime of a given bitsize to be found along with the reward and what a one year supply would look like mining

at the block target time speed of one block every 30 minutes.

$\|N\|_2$	$\|p_1\|_2$	Reward	1 Year New Supply
192	96	0.89429	15625.03331
256	128	1.78858	31250.06679
320	160	3.57716	62500.13375
384	192	7.15431	125000.08877
448	224	14.30862	250000.17771
512	256	28.61723	500000.17669

Figure 6: FACT0RN Blockchain Coin Supply at Different Difficulty Levels.

The following table shows the parameters for scrypt. This hashing algorithm is designed to be memory intensive and consume resources on an ASIC in such a way it cannot be optimized [6].

Parameter	Value
salt	version || $nBits$ || time
password	Prev Hash || $(\text{Txn})_r$ || nonce
N	2^{12}
r	2
p	1
Digest	2048-bits

Figure 7: Scrypt Parameters. RAM usage: 1MB.

The following pseudo-code is the replacement of SHA2 in bitcoin. It describes how a new block is mined and what checks are performed to validate a block.

1. **Input:** $nBit$:= Number of bits of N
2. **Output:** Boolean, accepted or not.
   (a) W =gHash(blocktemplate) //nBit pseudo-random number using block header.
   (b) Compute $\tilde{n} = 16 \cdot |W|_2$
   (c) Generate candidates $S = \{n \in \mathbb{N} \mid |W - n| < \tilde{n}\}$
   (d) for n in S:
   (e) P = factor(n)
   (f) if $|P| == 2$:
   (g) if $|P[0]|_2 == |P[1]|_2$:
   (h) if $P[0]$ is prime:
   (i) if $P[1]$ is prime:
   (j) return True
   (k) return False

Figure 8: Consensus Algorithm Core. (PoW)

The gHash function is quite long and intricate. Instead of writing pseudo-code that would take over a page I will explain the motivation for it and a brief technical overview.

The purpose of gHash is to protect the blockchain. Two attacks are noteworthy: pre-chosen semi-prime attacks and mining ASICs. It must be cheaper to factor the numbers produced by gHash than trying to break gHash. Similarly, gHash is designed to do grunt work. Building ASICs for it would be to incentivize very poor use of planetary resources.

Let's discuss ASICs for a moment: the goal of FACT0RN is to promote the study, advancement and development of new mathematics. In particular, if ASICs will be built in the future for this blockchain FACT0RN encourages that they be built for factoring, perhaps implementing new discoveries in math, as oppose to performing grunt work like hashing.

The FACT0RN has been designed for mining numbers as big as 2048-bit. Should this number be changed for any reason, a simple version change can get the job done. Perhaps

needing a hard fork, but doable nonetheless. Now the technical part, here are some details gHash:

1. SHA3-512
2. Scrypt using parameters for 1MB RAM.
3. Whirlpool
4. Shake2b
5. Finding 512-bit primes.
6. Modular Exponentiation of pseudo-random 512-bits primes.
7. Modular Inverses Modulo pseudo-random 512-bit primes.
8. Internal Rounds depend on the population count of previous hashes.
9. Branching in main loop.

Items 5, 6 and 7 were initially 2048 just like the blockchain design limit, but due to a practical limitation it imposes it was tweaked to be 512 bits. Namely, every time the factornd daemon is started it verifies all the blocks from a known point onwards and the initial 2048-bit parameters meant every time the daemon runs it would take an unreasonably long time to verify blocks since the last check point.

gHash runs in about $5-10$ milliseconds on x86; tested on both Intel and AMD processors. Implementing this function on an ASIC would be a nightmare, and it is designed to be. The resource consumption is vast in logic and memory.

In addition to the technical aspects that make ASIC gHash implementation expensive there is a protocol design decision that makes it worse. Namely, gHash produces a pseudo-random integer of the desired bit length and miners are allowed to find a semiprime anywhere within a distance $\tilde{n}$ of W; for every gHash output miners can expect to find about 200 semiprimes to search for an admissible semiprime. Heuristically, after sieving all prime factors $< 2^{26}$ from the candidate set S described in Figure 8 as well as the primes in set S itself there tend to remain about $160 - 240$ candidates.

The gHashing-to-factoring ratio is heavily weighted in favour of factoring to make the opportunity cost of gHashing so much more computationally profitable as to disincentivize gHash optimization.

5 Circumvent Factoring

Now the question becomes: is there a way of solving for a block other than factoring? As we have the low $\approx 5\%$ binary digits or so to alter in any way we see fit, is there a way to meet the requirements of the blockchain without factoring? The answer is, yes. Note that this $\approx 5\%$ decreases as the difficulty level of the blockchain increases.

There is a method in Marc Joye's paper "RSA Moduli with a Predetermined Portion:Techniques and Applications" [3] which gives us an algorithm and some heuristics about this. Algorithm 3 in section 4.1. In particular, this method will always find a solution if we can alter at least 33% of the bits in the lower part, instead of the 5% or lower we are allowed in the starting difficulty level of FACT0RN. Any percentage lower than that 33%, and heuristically, we may not find any solution; the lower the percentage of low bits that may be changed the lower the odds of finding a solution. This means any would-be solver would have to sample from gHash outputs more often until a solution is found.

A result of Landau from 1919 is that there are approximately $x\log(\log(x))/\log(x)$ semiprimes up to x. So, in an interval of length $2\tilde{n}$ we have an approximation of how many semiprimes we can expect:

$$\tau(x,\tilde{n}) = \frac{(x+\tilde{n})\log(\log(x+\tilde{n}))}{\log(x+\tilde{n})} - \frac{(x-\tilde{n})\log(\log(x-\tilde{n}))}{\log(x-\tilde{n})}$$

Now we would like to know how many of these semiprimes contain what we will call strong semiprimes. A semiprime is strong if both prime factors have the exact same number

of binary digits. We will need to sieve the semiprimes in this interval to determine if there are any strong semiprimes as those are the only ones the FACT0RN Blockchain will accept.

$\lvert W\rvert_2$	$\tilde{n}$	$\tau(W, \tilde{n})$
64	960	172
128	1920	205
192	2880	224
256	3840	238
320	4800	248
384	5760	257
448	6720	264
512	7680	270
576	8640	276
640	9600	281

Figure 9: Expected semiprimes in each interval at each bitsize.

Out of the expected number of semiprimes we need to find one such that their prime factors have the same number of binary digits. While these numbers are within the margin of error of the semiprime counting function it is the best approximation we have. As was noted earlier, after sieving primes up to 2^{26} and primes in the set S, these estimates are reasonably correct.

The first step in Method 3, referenced above, for our use case would be to pick a random prime number of $\frac{|W|_2}{2}$ binary digits. Suppose that the 200 semiprimes or so we get from sieving the range around every gHash output is in fact a strong semiprime — this is far from the truth, but let's take the best case scenario. What are the chances we pick a prime number that is a factor for any of these 200 or so strong semiprimes?

For example, the blockchain testnet starts at $|W|_2 = 210$, we would have to choose any one of the 400 or so right primes out of all primes with 105 binary digits: this probability is $\approx 400/10^{28} < 1/10^{25}$ using the prime counting function of x/ln(x). And notice, this is the easy case, $|W|_2$ starts at 230 for mainnet and gets bigger from there. Semiprimes are much more scarce in the $\tilde{n}$ interval around W than the assumption we just made. Several gHashes are expected to be needed to find one solution to a block.

There is one hybrid attack using both ASIC mining and Algorithm 3 to get a pre-chosen semiprime. We may sample gHash to get as many bits as we need and together with the free bits from $\tilde{n}$ form a pre-chosen strong semiprime.

For example, $\tilde{n}$ may only allows us to change about the low 12-bits at a difficulty level of 210-bits. But, for a difficulty level of 210-bits if we can choose 33% of the low bits we can use Algorithm 3 to obtain an admissible semiprime without factoring. That would mean we need to set the low 69 bits to values of our choosing. From $\tilde{n}$ we already have the lowest 12 bits for free. We can now sample gHash until we get the desired bit positions, from bit 13 to bit 69, and then set the low 12 bits to anything we want.

While this works, gHash is so computationally expensive that it would take one Intel Skylake CPU core at 4 GHz about $2^{56} \cdot 0.005$ seconds to complete. The question is, how many CPUs would be needed to finish that in 29 minutes? Just in time to submit it in the 30 minute block target time because we need some time run algorithm 3. Let's say running Algorithm 3 takes a minute. It would take $2^{56} \cdot 0.005/(1740 \text{ Seconds}) \approx 2^{37.59}$ CPUs.

Now the question is, how much computational power is that? Well, if each CPU had a cost of one dollar that many CPUs would cost about $10^{11.3}$ or ≈ 100 billion dollars. Note that this is the analysis for the starting difficulty level and that the cost rises exponentially with the increase of the difficulty level. And finally, about 2^{38} megabytes would be needed which amounts to about 128 petabytes of memory. A conservative estimates places this attack at 100 billion dollars on compute alone, plus another $2^{28} \approx 256$ million dollars at 1 dollar per gigabyte of RAM.

This hybrid attack is, now and for the foreseeable future, unlikely to be improved enough even with ASICs to be practical or profitable.

6 Future Work

There is some amount of work to be done to implement the FACT0RN deadpool feature in addition to the standard blockchain maintenance work.

The new opcode OP_DIVCHECK is a multi-precision integer arithmetic opcode that will take two inputs N and P to return 1 if $N \mod P \equiv 0$ and 0 otherwise, where P is not a trivial factor, i.e. $P \notin \{1, N\}$. This opcode will allow anyone to submit a number N to the deadpool. Submitting a transaction where the ScriptSig is one of the non-trivial factors P will unlock the funds in that deadpool transaction by running OP_DIVCHECK(N, P) to validate said P.

The cost of using this opcode will be significant compare to a standard transaction on the blockchain given how expensive it is to perform this operation.

On a related note about new code, because the PoW is very different from hashing FACT0RN requires the codebase to be updated in many more ways than previous bitcoin

forks; the fundamental changes have been done already but more than half of all the testing needs to be re implemented for this new PoW of factoring integers.

We expect there to be at least 1 hard fork planned for the FACT0RN Blockchain as development and improvements are worked on. It is entirely possible for a scheme to be devised to secure OP_DIVCHECK against miner-in-the-middle and re-organization attacks with the current opcode set in such a way that a hard fork is not required, but until such a method is found let us plan for at least one hard fork.

A final note on hard forks. It is my expectation that as the mathematical and cryptographic community get involved in this project there will be significant improvements, suggestions and new features proposed by creative technical folks. Should such proposals arrive they will, with high likelihood, be considered and implemented even if it means we will have to do a hard fork; proposals that move towards concentrating more computational power into factoring as a group and rewarding appropriately so that the efficiency of the system as a whole improves will receive serious considerations.

7 Contribution Request

If you work on blockchain development and would like to join a project please consider joining the FACT0RN Blockchain. I have only modified the bare minimum from Bitcoin v22.0 to make the FACT0RN blockchain work, there is a lot of work and refactoring to be done. In addition, any proposals and ideas to improve the blockchain are welcomed. The project can be found here: https://github.com/FACToRN/FACToRN.

A note about my writing. I use "we" where it feels more natural to explain material and "I" when referring to claims, opinions, observations that are my own, and I feel this distinction is important to note.

Appendix A

This section is to give a full worked out example of mining a block. For this example, let say we make a call to "getblocktemplate" and we get the following block template to mine:

prev block hash	f25a2d60d6fd4d8fbfab555c401afca008e6d6a97240c99e96ba58601d3e87f5
nBits	230
merkle tree hash	8266deca6c65b39468e6fb8596869a231b9582ee3818d12ba7240cb126ebfb44
Version	0x04
Time	1652138720
Nonce	0
wOffset	0
Factor	0

This is what a template block looks like. The red fields are determined by the blockchain itself, changing them will cause the blockchain to reject the block as invalid. The rose color fields can be changed by the miners but have some restrictions on the time, some voting mechanism on the version field and depending on which transactions were included or not the merkle tree hash changes.

The green field can be changed to any 64-bit value. We will talk about the blue fields last. Let's start mining from this template block, the first thing we need is to perform the gHash[2] function with this template block where the red, rose and green field will be used for hashing and produce a random integer of nBits= 230 bits.

Let's say gHash(block template) produces the integer below for the nonce value of 5:

$$W = 1178617138817682472276126179584462761998125423560374634408860637889401$$

Recall the FACToRN blockchain has the parameter $\tilde{n} = 16*|W|_2$. Since $|W|_2 = nBits = 230$ this means that $\tilde{n} = 16 * 230 = 3680$. Now we need to factor all the integers within a radius of 3680 from W to see if any of them is a strong semiprime.

Indeed, if we have an offset of -654 from W we get:

$$W - 654 = 317584604400785237069778885802448 83 \times 371119104164851735330905872739856 09$$

where the two factors have the same number of digits in binary. Now we submit the following block:

prev block hash	f25a2d60d6fd4d8fbfab555c401afca008e6d6a97240c99e96ba58601d3e87f5
nBits	230
merkle tree hash	8266deca6c65b39468e6fb8596869a231b9582ee3818d12ba7240cb126ebfb44
Version	0x04
Time	1652138720
Nonce	5
wOffset	-654
nP1 Factor	3175846044007852370697788858024488 3

Recall that the smallest prime factor of the two must be submitted or the block will be rejected. As you can see, the fields in blue act as a seal to make the block valid; they may be chosen to be any value by the miners but if nP1 does not divide $W + wOffset$ in such a way that the result is prime, of equal bit length to $nP1$ and not farther away than $\tilde{n}$ from W, then the block will be rejected. [3]

[2] See https://github.com/FACToRN/FACToRN/blob/main/src/pow.cpp#L209 for the implementation of gHash.

[3] See https://github.com/FACToRN/FACToRN/blob/main/src/pow.cpp#L98 for rejection conditions.

References

[1] EZ Blockchain. https://ezblockchain.net/smartbox/. April 09, 2022.

[2] Richard P. Brent. Some integer factorization algorithms using elliptic curves. 1985.

[3] Marc Joye. Rsa moduli with a predetermined portion: Techniques and applications. *Information Security Practice and Experience (ISPEC 2008)*, 4991:116–130, 2008.

[4] RSA Laboratoies. *RSA Challenge*. March 18, 1991, See https://en.wikipedia.org/wiki/RSA_Factoring_Challenge.

[5] A.K. Lenstra and Jr. H.W. Lenstra. *The development of the number field sieve*. Springer-Verlag, 1993.

[6] Colin Percival. Stronger key derivation via sequential memory-hard functions. 2009.

[7] Carl Pomerance. A tale of two sieves. *Notices of the AMS*, 43(12):1473–1485, 1996.

[8] Bitcoin Uses More Electricity Than Many Countries. How Is That Possible? https://www.nytimes.com/interactive/2021/09/03/climate/bitcoin-carbon-footprint-electricity.html. September 03, 2021.

[9] A. Shamir R.L. Rivest and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. 21:120–126, 1978.

[10] Coinbase Giving Trent Fuenmayor, Program Manager. Announcing our second developer grant winners. 2021, January 13. https://blog.coinbase.com/announcing-our-second-developer-grant-winners-ffd3f6e93860.

[11] user "rogue". https://mersenneforum.org/showthread.php?t=9611. 2007.